

算法思维反思日志

学生求职匹配智能体——基于大模型 Prompt 工程的算法实践

一、初始目标与思路

这个项目的最初目标是做一个纯前端的网页工具：用户粘贴简历文本和岗位描述，点击一个按钮，就能拿到大模型给出的匹配度评分、差距分析和优化建议。之所以选择纯前端方案，是因为目标用户是学生群体，不需要注册登录，打开网页就能用。

核心思路是"Prompt 即算法"——不写传统的评分函数或规则引擎，而是把整个分析逻辑编码到一段精心设计的 Prompt 模板中。Prompt 需要同时做到三件事：界定角色（求职顾问专家）、约束输出格式（固定四个章节）、给出示例（Few-shot）引导模型按预期结构返回结果。

二、任务拆解与迭代过程

整个开发过程经历了三次关键的 Prompt 迭代。

第一版 Prompt：只写了角色设定和四个任务描述，没有约束输出格式。结果模型返回了大量 Markdown 表格、加粗符号和随机的编号方式，导致前端根本无法解析。

第二版 Prompt：加入了"严格按照下方格式输出"的指令，并写了示例。这次格式统一了，但模型有时会在评分理由里夹杂代码块，或者在差距分析前插入"根据您的简历，我发现以下问题"之类的前置废话，导致提取逻辑出错。

第三版 Prompt（最终版）：新增了三句关键约束："不要输出 Markdown 表格、加粗符号（**）或 HTML 标签"——这句话是经过四次测试后提炼的，一次列全三种常见干扰格式，比分别说明更有效。同时加入"每条不超过 30 字"的长度限制，有效抑制了模型对差距分析的过度展开。

对应前端解析层也做了三层容错：先用正则清洗 Markdown 残留，再用多组正则按优先级匹配评分，最后一层用关键词定位法切割四个章节并提取列表项。

三、大模型的表现评估

表现好的方面：加了 Few-shot 示例后，格式遵循度从约 60% 提升到接近 95%。特别是在“差距分析”和“优化建议”两个列表型章节，模型能稳定输出以“-”开头的条目，解析器几乎不需要容错。评分值与人工判断的相关性也较高，逻辑大体合理。

跑偏的地方：当输入的简历或岗位描述非常短时，模型倾向于给出偏高的评分（如 90+），理由则变得空洞。另一个问题是模型偶尔会用半角冒号替代全角冒号，导致中英文标点混用，正则需要同时覆盖两种锚点。

四、问题与修复策略

第一个问题是浏览器 CORS 跨域限制——本地直接用 fetch 调用大模型 API 会被浏览器拦截。最初尝试了三个公共 CORS 代理（corsproxy.io、allOrigins、cors-anywhere），但都极不稳定，频繁返回 502。最终方案是在 CloudBase 上部署一个云函数作为 HTTP 代理，通过 callFunction() 转发请求。这个方案最有效，因为云函数与前端同域，彻底绕过了浏览器的安全限制。

第二个问题是 CDN 缓存导致代码版本不一致。上传新版 app.js 后，由于缺少缓存破坏参数，CDN 仍返回旧文件，导致新 HTML 里的云函数代理选项不被旧代码识别，引发 parseFloat 返回 NaN 的 bug。修复方法很简单：给 script 引用加上 ?v=2 版本号参数。但这个过程让我意识到，前端的“版本一致性”本身就是一个需要设计的算法环节。

五、最终总结

这次实践让我对“算法思维”的理解发生了根本变化。过去我认为算法就是排序、搜索、动态规划等经典范式，但现在我认识到，在大模型时代，Prompt Engineering 本身就是一种高层次的算法设计——它不操作数据结构的指针，而是操作模型的注意力分布。一个好的 Prompt 需要像写伪代码一样精准：定义边界、处理异常、给测试用例。

更重要的是，"算法"不再止步于模型输出，而是延伸到了前端的解析层、请求的传输层、甚至部署的缓存层。整个系统每个环节都需要容错设计——从 Prompt 的格式约束，到正则的多模式匹配，再到 CORS 的代理降级和 CDN 的缓存失效。

六、作品的真实价值

我会继续使用这个工具。实际帮身边找实习的同学测过几份简历，匹配度评分确实能提供一个客观参考，差距分析也比泛泛的"改改简历"具体得多。下一步想加两个功能：一是接入 PDF 解析让用户直接上传简历文件，二是支持多岗位同时对比，输出排名。

这个项目让我确信：在 LLM 应用中，真正的工程价值不在于调用了哪个模型，而在于你如何用 Prompt 把模糊的"帮我看看"转译成可解析、可展示、可复用的结构化结果。